

本次课程内容

- 文件操作
 - 文件的打开方式
 - 文件读写，文件结束判断
 - 文件的定位
- 黄容图像处理程序

第 13 章 文件

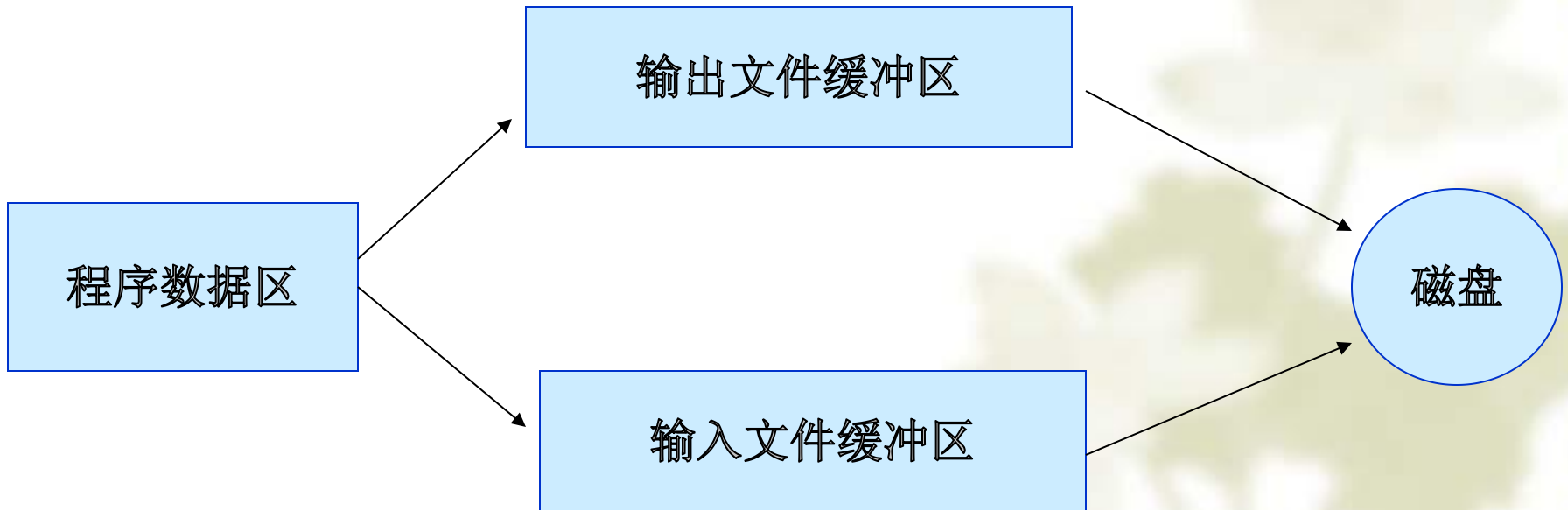
- 文件的用途
 - ✎ 保存数据处理的中间或者最终结果；大容量、断电后不丢失。
- 广义的概念
 - ✎ 包括键盘、打印机、显示屏
- 类型
 - ✎ 文本文件：信息以 **ASCII** 码方式存放。优点：便于人们阅读。
缺点：占用空间多、处理前后需要类型转换。
 - ✎ 二进制文件：信息以二进制方式存放。优缺点相反。
 - ✎ 对应着不同的函数
- 数字的存储
 - ✎ 二者的区别甚大

整数的两种存放方式

存放类型	所要存放的数据(10进制)	文件中的数据(16进制)	在编辑器中看到的
ASCII	129	31 32 39	129
二进制	129	81 00	ü

文件缓冲区的概念

- 目的：减少读取外部存储设备的次数
- 程序员“看不到”这两个缓冲区



FILE 结构体

- 存放文件操作过程中的状态信息
- 程序员一般可以不管其中的细节。
- 但是：所有的文件操作函数都需要此信息。精确地说：需要指向此信息的一个指针（文件句柄）
- 格式： **FILE * f** ; **FILE * f[5]**;
- 文件操作的三个步骤

一个完整的例子

```
#include <stdio.h>
```

```
int main( )
```

```
{
```

```
    FILE * f;
```

```
    f=fopen("tem.txt", "w");
```

```
    fprintf(f, "A number is to be written %d", 123);
```

```
    fclose(f);
```

```
}
```

文件的打开

- 格式:

`FILE *f; f=fopen(文件名, 使用方式)`

- 例子: `f=fopen("a1", "r");`

↪ 文件名字 “d:\\tem\\data.txt”

↪ 使用方式

↪ 句柄

r : 只读，不可写，文件不存在时返回NULL。

w : 只写，不可读，文件不存在时创建新文件，
文件存在时原有内容被清除。

a : 只写，不可读，文件不存在时创建文件，
文件存在时原有内容保留，**所有写操作只发
生在文件尾！** 总之：添加数据。

r+: 可读写，文件不存在时返回NULL。

w+: 可读写，文件不存在时创建新文件，
文件存在时原有内容被清除。

a+: 可读写，文件不存在时创建新文件，文件存在
时原有内容保留。可在任意位置读写。初始
指针位置：文件末尾

rb, wb, ab, rb+, wb+, ab+ : 二进制版

基本的“a”方式操作

```
#include <stdio.h>
int main()
{
    FILE *f;
    if ( (f=fopen("data.txt", "a"))==NULL)
        printf("fopen error\n");    return -1;
}

fprintf(f, "this is a newline\n");
fclose(f);
}
```

“a”方式不可以读

```
#include <stdio.h>

int main()
{
    FILE *f;
    char aline[200];

    if ( (f=fopen("data.txt", "a"))==NULL) {
        printf("fopen error\n");  return -1;
    }

    fseek(f, 0L, SEEK_SET);
    fgets(aline, sizeof(aline), f);    // wrong!
    printf("%s", aline);

    fprintf(f, "this is a newline\n");
    fclose(f);
}
```

```
#include <stdio.h>
int main()
{
    FILE *f;
    char aline[200];

    if ( (f=fopen("data.txt", "a"))==NULL) {
        printf("fopen error\n");    return -1;
    }
    fseek(f, 0L, SEEK_SET);

    fprintf(f, "this is a newline\n");
    fclose(f);
}
```

“a” 方式所有操作只发生在文件尾

“r+”方式可读写

```
#include <stdio.h>

int main()
{
    FILE *f;
    char aline[200];
    if ( (f=fopen("data.txt", "r+"))==NULL) {
        printf("fopen error\n");    return -1;
    }
    fseek(f, 0L, SEEK_SET);
    fgets(aline, sizeof(aline), f);
    puts(aline);

    fseek(f, 0L, SEEK_END);
    fprintf(f, "this is a newline\n");
    fclose(f);
}
```

“w+”方式可读写

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *f;
```

```
    char aline[200];    int i;
```

```
    if ( (f=fopen("data.txt", "w+"))==NULL) {
```

```
        printf("fopen error\n");    return -1;
```

```
    }
```

```
    for (i=0; i<5; i++)
```

```
        fprintf(f, "this is line %d\n", i);
```

```
    fseek(f, 0L, SEEK_SET);
```

```
    for (i=0; i<5; i++) {
```

```
        fgets(aline, sizeof(aline), f);
```

```
        puts(aline);
```

```
    }
```

```
    fclose(f);
```

```
}
```

“a+”方式可读写

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *f; int i;
```

```
    char aline[200];
```

```
    if ( (f=fopen("data.txt", "a+"))==NULL) {
```

```
        printf("fopen error\n");    return -1;
```

```
    }
```

```
    for (i=0; i<5; i++)
```

```
        fprintf(f, "hello %d\n", i);
```

```
    fseek(f, 0L, SEEK_SET);
```

```
    for (i=0; i<5; i++) {
```

```
        fgets(aline, sizeof(aline), f);
```

```
        puts(aline);
```

```
    }
```

```
    fclose(f);
```

```
}
```

文件的关闭

- 格式 **fclose**(文件指针)
- 目的：结束对一个文件的操作。
缓冲区
文件指针不再有效。
- 记住关闭文件！

```
#include <stdio.h>
```

```
int main()
```

```
{ FILE *f;
```

```
  char ch;
```

```
  if ( (f=fopen("data.txt", "a+")) == NULL) {
```

```
    printf("fopen error\n"); return -1;
```

```
  }
```

```
  ch=fgetc(f);
```

```
  while ( ch!=EOF) {
```

```
    putchar(ch);
```

```
    ch=fgetc(f);
```

```
  }
```

```
  fclose(f);
```

```
}
```

文件结束判定
仅仅针对文本文件


```
#include <stdio.h>
```

```
int main()
```

```
{ FILE *f1, *f2;
```

```
  char ch;
```

```
  if ( (f1=fopen("data.bin", "r"))==NULL) {  
    printf("fopen error\n");  return -1;  
  }
```

```
  if ( (f2=fopen("data.dst", "w"))==NULL) {  
    printf("fopen error\n");  return -1;  
  }
```

```
  for (ch=fgetc(f1); ch!=EOF; ) {  
    fputc(ch, f2);  ch=fgetc(f1);  
  };
```

```
  fclose(f1);  fclose(f2);
```

```
}
```

上述文件结束判定
不适用于二进制文件

```
#include <stdio.h>
```

```
int main()
```

```
{ FILE *f1, *f2;
```

```
  char ch;
```

```
  if ( (f1=fopen("data.bin", "rb") ) == NULL) {  
    printf("fopen error\n");    return -1;  
  }
```

```
  if ( (f2=fopen("data.dst", "wb") ) == NULL) {  
    printf("fopen error\n");    return -1;  
  }
```

```
  for (ch=fgetc(f1); !feof(f1); ) {  
    fputc(ch, f2);  
    ch=fgetc(f1);  
  };
```

```
  fclose(f1);    fclose(f2);
```

```
}
```

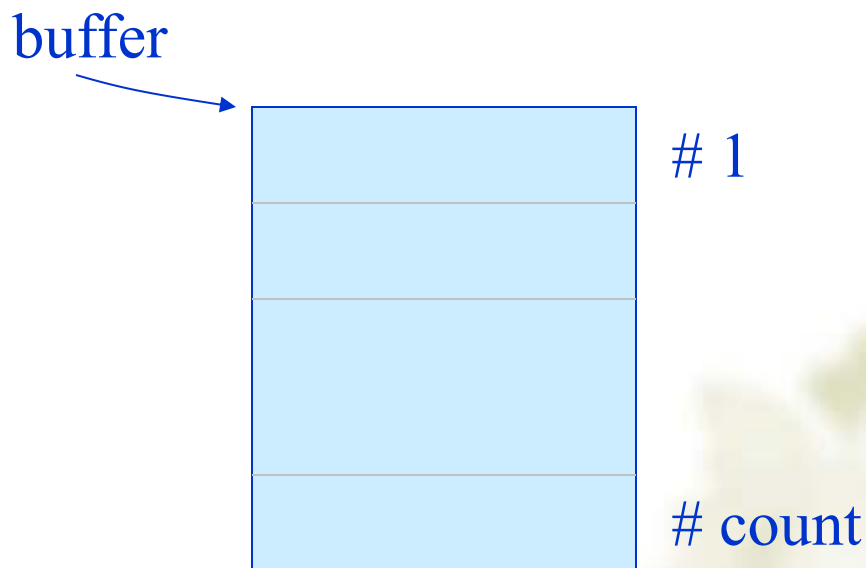
二进制文件的结束判定

feof 的讨论

- 对文本方式、二进制方式打开的文件，具有不同的含义：
 - ✧ 文本方式打开：判断是否读到文件结束符。
如果是，为真。如果以文本方式打开一个二进制文件，**feof** 会出错。
 - ✧ 二进制方式打开：判断是否已经把文件的所有内容读完。
如果是，为真。
- 结论：只要文件的打开方式和文件的实际类型一致，则 **feof** 正确执行。

二进制文件的读写操作

```
fread(void *buffer, size_t size, size_t count, FILE *fp );  
fwrite(void *buffer, size_t size, size_t count, FILE *fp );
```



```
#include <stdio.h>
#define SIZE 4
struct student_type {
    char name[10];
    int num;
    int age;
    char addr[15];
} stud [SIZE];
```

```
int main()
{
    int i;
    FILE *fp;
    for (i=0; i<SIZE; i++)
        scanf("%s %d %d %s", stud[i].name,
            &stud[i].num, &stud[i].age, stud[i].addr);

    if (( fp=fopen("stu_list", "wb") )==NULL) {
        printf("can not open file\n");
        return -1;
    }

    for (i=0; i<SIZE; i++)
        if (fwrite(&stud[i], sizeof(struct student_type), 1, fp) !=1 )
            printf("file write error\n");

    fclose(fp);
}
```

```
#include <stdio.h>
#define SIZE 4
struct student_type {
    char name[10];
    int num;
    int age;
    char addr[15];
} stud[SIZE];
```

```
int main()
{
    int i;
    FILE *fp;

    fp=fopen("stu_list", "rb");

    for (i=0; i<SIZE; i++) {
        fread(&stud[i], sizeof(struct student_type), 1, fp);

        printf("%s %d %d %s\n", stud [ i ].name,
            stud[ i ].num, stud[ i ].age, stud[ i ].addr);
    }

    fclose(f);
}
```


fprintf 函数和 fscanf 函数

- 格式:

fprintf(文件指针, 格式字符串, 输出列表);

fscanf(文件指针, 格式字符串, 输入列表);

- 特点:

- ✧ 以文本文件方式存储

- ✧ 输入输出的时候速度慢

```
#include <stdio.h>
```

```
struct student_info {  
    int number;  
    char name[80];  
    int gender;  
    float score;  
} s={123, "zhang_san", 1, 78.0};
```

```
int main()  
{  
    FILE *fp;
```

```
if (( fp=fopen("tem.dat", "w") )==NULL){  
    printf("can not open file\n");  
    return -1;  
}  
fprintf(fp, "%d %s %d %f", s.number,  
        s.name, s.gender, s.score);  
fclose(fp);  
  
if (( fp=fopen("tem.dat", "r") )==NULL)  
{  
    printf("can not open file\n");  
    return -1;  
}
```

```
fscanf(fp, "%d %s %d %f", & s.number,  
        s.name, & s.gender, & s.score);
```

```
fclose(fp);
```

```
printf("%d %s %d %f", s.number,  
        s.name, s.gender, s.score);
```

```
}
```

其他读写函数

- **char *fgets(char *string, int n, FILE *stream);**

☞ n 一般为 str 的大小

☞ 最多只从 fp 中读入 n - 1 个字符。

- **int fputs(const char *string, FILE *stream);**

- putw, getw 函数

☞ 输入、输出整数，用的少。

文件的定位

- 文件位置指针（file position indicator）的概念
- 函数 `fseek(FILE *fp, long offset, int whence);`
whence: `SEEK_SET`, `SEEK_CUR`, `SEEK_END`
- 函数 `ftell(FILE *fp)`
- 函数 `rewind(FILE *fp)`

```
#include <stdio.h>
int main()
{
    FILE *f;
    int i, j;

    f=fopen("tem.dat", "wb+");
    for (i=0; i<100; i++)
        fwrite(&i, sizeof(i), 1, f);
    for (i=0; i<100; i+=2){
        fseek(f, (long)i*sizeof(int), SEEK_SET);
        fread(&j, sizeof(j), 1, f);
        printf("%d ", j);
    }
    fclose(f);
}
```

我们的成果：处理黄蓉图像的C程序

```
/* The input image is in RAW format. The whole file has  
three sections: R section, G section and B section.  
Within each section, one byte is used to represent the  
color.
```

```
*/
```

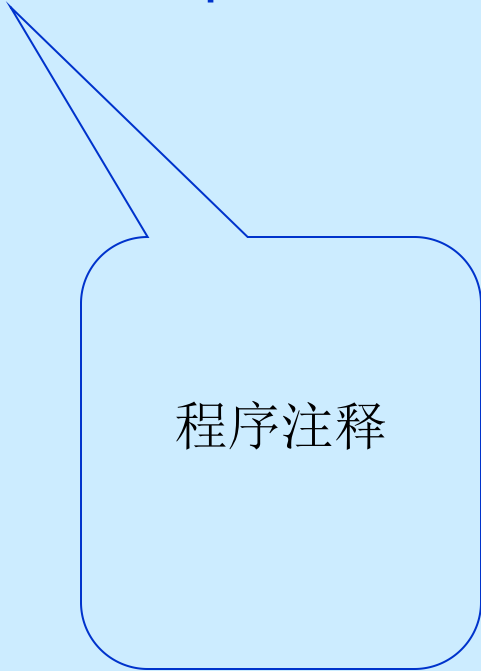
```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define W 300
```

```
#define H 400
```

```
/* to be continued */
```



程序注释


```
void sort(char x[], int n)
{
    int i,j,k;
    char t;
    for (i=0; i<n-1; i++) {
        k=i;
        for (j=i+1; j<n; j++)
            if (x[j]>x[k]) k=j;
        if ( k!=i) {
            t=x[i]; x[i]=x[k]; x[k]=t;
        }
    }
} /* to be continued */
```

```
void medfilt2(char * p1, char * p2, int width, int heigh)
{  int i, j, offset;
   char colors[9], *p;
   for (i=2; i<heigh-2; i++)
       for (j=2; j<width-2; j++){
           offset = i * width + j; p=p1+offset-width-1;
           colors[0]=*p++; colors[1]=*p++; colors[2]=*p;
           p=p1+offset-1;
           colors[3]=*p++; colors[4]=*p++; colors[5]=*p;
           p=p1+offset+width-1;
           colors[6]=*p++; colors[7]=*p++; colors[8]=*p;
           sort(colors, 9); *(p2+offset)=colors[4];
       }
}/* to be continued */
```

```
int main()
{  FILE *f1, *f2;
   char * p1, *p2;
   long fsize;

   f1=fopen("hr1-n.raw", "rb");
   f2=fopen("hr1-c.raw", "wb");
   if (f1==NULL || f2==NULL) {
       printf("file open error\n"); exit(-1);  };
   fseek(f1, 0, SEEK_END);          fsize=ftell (f1);
   fseek(f1, 0, SEEK_SET);
   p1=(char *) malloc(fsize);
   p2=(char *) malloc(fsize);
   if (p1==NULL || p2==NULL) {
       printf("memory error\n");  exit(-1);
   };
};
```

```
fread(p1, fsize, 1, f1);
```

```
medfilt2(p1, p2, W, H);
```

```
medfilt2(p1+W*H, p2+W*H, W, H);
```

```
medfilt2(p1+W*H*2, p2+W*H*2, W, H);
```

```
fwrite(p2, fsize, 1, f2);
```

```
free(p1); free(p2);
```

```
fclose(f1); fclose(f2);
```

```
}
```

Corrupted by the
“Salt and pepper” noise



中值滤波后的图像

